

Escalonador por Compartilhamento Proporcional de CPU

Lottery Scheduler no xv6

Disciplina: Sistemas Operacionais

Lucas Dolabella de Castro Lopes - 2023001549

Vanessa Nascimento Silva - 2023001816

2026

1. Introdução

Este trabalho consiste na implementação de um Escalonador por Loteria no sistema operacional xv6. O objetivo é substituir o escalonador round-robin original por um algoritmo proporcional: cada processo recebe um número de tickets e, a cada ciclo de escalonamento, um sorteio determina qual processo será executado. Processos com mais tickets têm maior probabilidade de serem selecionados, recebendo assim uma fatia proporcional do CPU.

Foram implementadas duas novas chamadas de sistema: `settickets(int n)`, que define o número de tickets do processo chamador, e `getpinfo(struct pstat*)`, que retorna informações sobre todos os processos em execução. Além disso, o escalonador foi modificado para utilizar um gerador de números pseudo aleatórios e realizar o sorteio a cada decisão de escalonamento.

2. Modificação 1 - Estrutura de Dados para Tickets

2.1. Descrição

Para rastrear o número de tickets de cada processo, foram adicionados dois novos campos a estrutura `struct proc` em `kernel/proc.h`: o campo `tickets` armazena quantos tickets o processo possui, e o campo `ticks` conta quantas vezes o processo foi selecionado pelo escalonador. Adicionalmente, foi criado o arquivo `kernel/pstat.h` com a estrutura `pstat` exigida pelo enunciado, e a constante `DEFAULT_TICKETS = 1` foi adicionada em `kernel/param.h` para definir o valor inicial de tickets de cada novo processo.

2.2. Diff - `kernel/param.h` e `kernel/proc.h`

```
diff --git a/kernel/param.h b/kernel/param.h index
cd2a41c..533168f 100644
--- a/kernel/param.h
+++
b/kernel/param.h
h @@ -12,4
+12,5 @@
#define FSSIZE 2000 // size of file system in
blocks #define MAXPATH 128 // maximum file
path name #define USERSTACK 1 // user stack
pages
+#define DEFAULT_TICKETS 1 // default lottery tickets per process

diff --git a/kernel/proc.h b/kernel/proc.h index
d021857..21a0a17 100644
--- a/kernel/proc.h
+++ b/kernel/proc.h
@@ -104,4 +104,6 @@ struct proc {
    struct file *ofile[NOFILE]; // Open files
    struct inode *cwd; // Current directory
```

```

    char name[16];                // Process name (debugging)
+ int tickets;                   // lottery tickets
+ int ticks;                     // times scheduled by lottery
};

```

2.3. Diff - kernel/pstat.h (arquivo novo)

```

diff      --git      a/kernel/pstat.h
b/kernel/pstat.h    new    file    mode
100644
index 00000000..c334b9f
--- /dev/null
+++
b/kernel/pstat.
h @@ -0,0 +1,13
@@
+#ifndef _PSTAT_H_
+#define _PSTAT_H_
+
+#include "param.h"
+
+struct pstat {
+ int inuse[NPROC];    // 1 if slot is in use, 0 otherwise
+ int tickets[NPROC]; // number of tickets this process has
+ int pid[NPROC];     // PID of each process
+ int ticks[NPROC];   // number of ticks each process has accumulated
+};
+
+#endif

```

O campo tickets é lido pelo escalonador a cada ciclo para calcular o total de tickets entre processos RUNNABLE. O campo ticks é incrementado toda vez que o processo vence o sorteio, servindo como métrica de uso de CPU. Ambos os campos são inicializados em allocproc() e zerados em freeproc() quando o slot é liberado.

3. Modificação 2 - Gerador de Números Aleatórios e Escalonador

3.1. Gerador de Números Pseudo Aleatórios (LCG)

O xv6 não possui uma função rand() nativa. Foi implementado um Gerador Congruente Linear (LCG - Linear Congruential Generator) diretamente em kernel/proc.c. O LCG utiliza a fórmula: estado_novo = estado_atual * 1664525 + 1013904223. O estado inicial é 1 e evolui a cada chamada.

3.2. O Escalonador por Loteria

O escalonador original (round-robin) foi substituído pelo escalonador por loteria. O algoritmo funciona em duas passagens pelo vetor de processos: na primeira, soma todos os tickets dos processos em estado RUNNABLE (processos dormindo são automaticamente

excluídos pois não estão RUNNABLE); na segunda, percorre os processos acumulando tickets até ultrapassar o número vencedor sorteado.

3.3. Diff - kernel/proc.c

```
diff      --git      a/kernel/proc.c
b/kernel/proc.c      index
22a5401..c7708c2 100644
--- a/kernel/proc.c
@@ -12,6 +12,15 @@ struct proc
proc[NPROC]; struct proc *initproc;
+static uint rand_state = 1;
+
+static uint
+rand(void)
+{
+ rand_state = rand_state * 1664525 + 1013904223;
+ return rand_state;
+}
+
+int nextpid = 1;
+struct spinlock pid_lock;

@@ -124,6 +133,8 @@
allocproc(void) found:
    p->pid =
        allocpid();
    p->state =
        USED;
+ p->tickets = DEFAULT_TICKETS;
+ p->ticks = 0;

@@ -168,6 +179,8 @@ freeproc(struct
proc *p) p->chan = 0;
p->killed =
0;
p->xstate =
0;
+ p->tickets
= 0;
+ p->ticks = 0;
    p->state = UNUSED;
}

@@ -289,6 +302,7 @@
kfork(void) np->cwd =
idup(p->cwd);

safestrncpy(np->name, p->name, sizeof(p->name));
+ np->tickets = p->tickets;

@@ -429,35 +443,42 @@ scheduler(void)
```

```

c->proc = 0;
for(;;){
-   // The most recent process to run may have had interrupts
-   // turned off; enable them to avoid a deadlock if all
-   // processes are waiting. Then turn them back off
-   // to avoid a possible race between an interrupt
-   //
-   and
-   wfi.
-   intr
-   _on(
-   );
-   intr
-   _off
-   ();

-   int found = 0;
+   // Count total tickets among RUNNABLE processes.
+   int total = 0;

-   for(p = proc; p < &proc[NPROC];
-   p++) { acquire(&p->lock);
-   if(p->state == RUNNABLE) {
-   p->state = RUNNING;
-   c->proc = p;
-   swtch(&c->context, &p->context);
-   c->proc = 0;
-   found = 1;
-   }
+   if(p->state == RUNNABLE)
+   total +=
+   p->tickets;
-   release(&p->lock);
-   }

-   if(found == 0) {
-   // nothing to run; stop running on this core until an interrupt.
+   if(total == 0) {
+   asm volatile("wfi");
+   continue;
+   }

+   // Draw a winning ticket in [0, total).
+   int winner = rand() % total;
+   int counter = 0;

+   for(p = proc; p < &proc[NPROC]; p++) {
+   acquire(&p->lock);
+   if(p->state == RUNNABLE) {
+   counter += p->tickets;
+   if(counter > winner) {

```

```

+         p->state = RUNNING;
+         p->ticks++;
+         c->proc = p;
+         swtch(&c->context, &p->context);
+         c->proc = 0;
+         release(&p->lock);
+         break;
+     }
+ }
+ release(&p->lock);
+ }
+ }
+ }

```

4. Modificação 3 - Preenchimento da Estrutura pstat

4.1. Descrição

A chamada de sistema `getpinfo(struct pstat*)` preenche a estrutura `pstat` com informações sobre todos os processos. A implementação em `kernel/sysproc.c` percorre o vetor de processos `proc[]`, adquire o lock de cada processo, e os campos relevantes e preenche os arrays da estrutura. Ao final, utiliza `copyout()` para copiar os dados com segurança do espaço do kernel para o endereço de memória do processo usuário.

4.2. Diff - kernel/sysproc.c

```

diff --git a/kernel/sysproc.c
b/kernel/sysproc.c index 8e4a85a..1da434c
100644
--- a/kernel/sysproc.c
+++
b/kernel/sysproc
.c @@ -7,6 +7,7
@@
#include
"proc.h"
#include
"vm.h"
#include
"syscall.h"
+#include "pstat.h"

@@ -109,6 +110,44 @@
sys_uptime(void) return
xticks;
}
+uint64
+sys_settickets(void)
+{
+ int n;
+ argint(0, &n);
+ if(n < 1)

```

```

+     return -1;
+ myproc()->tickets = n;
+ return 0;
+}
+
+uint64
+sys_getpinfo(void)
+{
+ uint64 addr;
+ argaddr(0, &addr);
+ if(addr == 0)
+     return -1;
+
+ struct pstat ps;
+ extern struct proc proc[];
+ struct proc *p;
+ int i = 0;
+
+ for(p = proc; p < &proc[NPROC]; p++, i++) {
+     acquire(&p->lock);
+     ps.inuse[i] = (p->state != UNUSED) ? 1 : 0;
+     ps.tickets[i] = p->tickets;
+     ps.pid[i] = p->pid;
+     ps.ticks[i] = p->ticks;
+     release(&p->lock);
+ }
+
+ if(copyout(myproc()->pagetable, addr, (char *)&ps, sizeof(ps)) < 0)
+     return -1;
+ return 0;
+}

```

O lock de cada processo é adquirido individualmente durante a leitura para garantir consistência dos dados. A função `copyout()` verifica se o endereço de destino é válido no espaço de endereçamento do processo usuário antes de realizar a cópia, evitando vulnerabilidades de segurança.

5. Modificação 4 - Registro das Chamadas de Sistema

5.1. Descrição

Para expor as novas chamadas de sistema ao espaço do usuário, foram adicionados os números de syscall em `kernel/syscall.h`, registradas no vetor de despacho em `kernel/syscall.c`, e adicionadas as entradas de stub em `user/usys.pl` e os protótipos em `user/user.h`.

5.2. Diff - `kernel/syscall.h` e `kernel/syscall.c`

```

diff --git a/kernel/syscall.h
b/kernel/syscall.h index 72d3e4f..064bc3c

```

```

100644
--- a/kernel/syscall.h
+++ b/kernel/syscall.h
@@ -20,6 +20,8 @@
#define SYS_link 19
#define SYS_mkdir 20
#define SYS_close 21
-#define SYS_getcnt 22
+#define SYS_getcnt 22
+#define SYS_setticket 23
+
+#define SYS_getpinfo 24

-#define NSYSCALLS 23
+#define NSYSCALLS 25

diff --git a/kernel/syscall.c
b/kernel/syscall.c index 335d6ff..25a153e
100644
--- a/kernel/syscall.c
+++ b/kernel/syscall.c
@@ -102,6 +102,8 @@ extern uint64
sys_close(void); uint64 sys_getcnt(void);
+extern uint64 sys_settickets(void);
+extern uint64 sys_getpinfo(void);

@@ -127,7 +129,9 @@ uint64
(*syscalls[])(void) = { [SYS_link]
sys_link,
[SYS_mkdir]
sys_mkdir,
[SYS_close]
sys_close,
-[SYS_getcnt] sys_getcnt,
+[SYS_getcnt] sys_getcnt,
+[SYS_settickets] sys_settickets,
+[SYS_getpinfo] sys_getpinfo,
};

```

5.3. Diff - user/user.h e user/usys.pl

```

diff --git a/user/user.h
b/user/user.h index
4a123f6..0e97292 100644
--- a/user/user.h
+++
b/user/user.
h @@ -1,6
+1,7 @@
#define SBRK_ERROR ((char *)-1)

struct stat;
+struct pstat;

@@ -25,6 +26,8 @@ char*

```



```

sys_sbrk(int,int); int pause(int);
int
uptime(void
);      int
getcmt(int)
;
+int settickets(int);
+int getpinfo(struct pstat*);

diff      --git      a/user/usys.pl
b/user/usys.pl      index
6c91d3f..d534e3a 100644
--- a/user/usys.pl
+++ b/user/usys.pl
@@ -43,3 +43,5 @@ entry("sbrk");
entry("pause");
entry("uptime");
entry("getcmt");
+entry("settickets");
+entry("getpinfo");

```

6. Testes e Resultados

6.1. Teste de proporção 2:1 (ltest)

O programa `user/ltest.c` verifica o comportamento do escalonador com dois processos: o pai recebe 2 tickets (via `settickets(2)` antes do `fork`) e o filho recebe 1 ticket (via `settickets(1)` logo após o `fork`). O pai executa um laço de 500 milhões de iterações enquanto o filho gira indefinidamente. Ao final, `getpinfo()` é usado para comparar os ticks acumulados de cada processo.

Resultado obtido ao executar 'ltest' no xv6:

```

parent (2 tickets): 17 ticks
child (1 ticket):    6
ticks   ratio:    17:6
(expected ~2:1)

```

6.2. Teste de proporção 3:2:1 (graph)

O programa `user/graph.c` verifica o comportamento com três processos simultâneos com 30, 20 e 10 tickets respectivamente. O pai cria os três filhos, cada um com seu número de tickets definido via `settickets()` antes do `fork`. Após criar os três filhos, o pai chama `settickets(1)` para reduzir sua própria participação no sorteio, evitando interferência significativa na distribuição de CPU entre os filhos. A cada 50 ticks do sistema, `getpinfo()` é chamado e os ticks ganhos por cada processo no intervalo são impressos.

Resultado obtido ao executar 'graph' no xv6:

sample	A(30)	B(20)	C(10)
1	55	41	26
2	34	18	8
3	30	21	11
4	84	49	27
5	19	16	15
6	38	18	16
7	34	24	12
8	38	17	10
9	52	23	15
10	24	12	4
total	408	239	144

Os totais 408:239:144 correspondem aproximadamente à razão 2.8:1.7:1, muito próxima da razão esperada de 3:2:1. A diferença é explicada pela natureza probabilística do algoritmo e pelo curto tempo de execução do teste.

6.3. Gráfico - Distribuição de CPU ao longo do tempo

O gráfico abaixo foi gerado pelo script plot graph.py, que inicia o xv6 automaticamente via QEMU, executa o programa graph, captura a saída e plota os resultados com matplotlib.

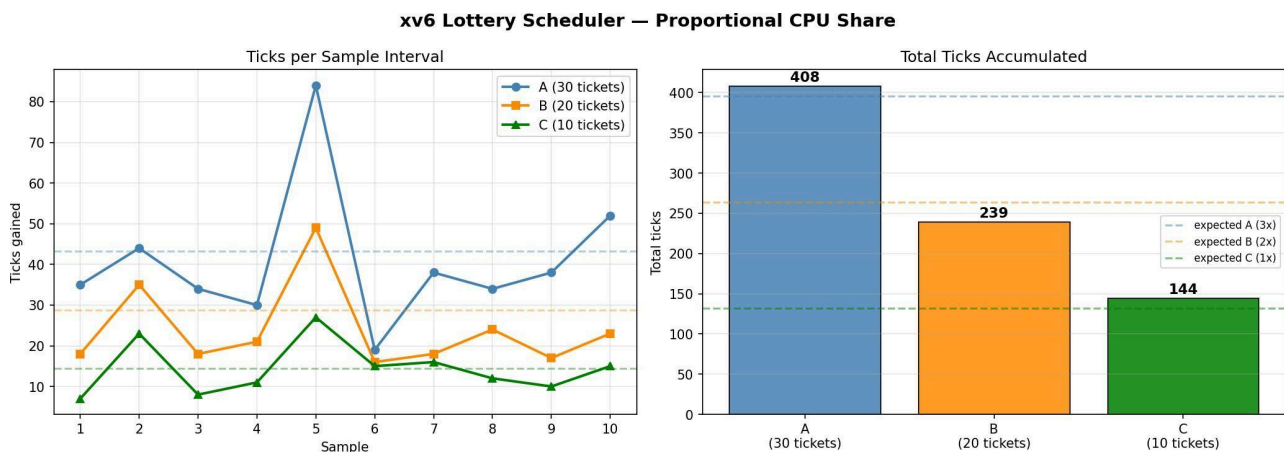


Figura 1: Ticks por intervalo (esq.) e totais acumulados (dir.) para processos com 30, 20 e 10 tickets.

7. Conclusão

O Escalonador por Loteria foi implementado com sucesso no xv6. As principais modificações realizadas foram: adicao dos campos tickets e ticks ao struct proc,

implementação de um gerador de números pseudo aleatórios (LCG) no kernel, substituição do escalonador round-robin pelo algoritmo de loteria, implementação das chamadas de sistema settickets() e getpinfo(), e criação da estrutura pstat para exposição de dados ao espaço do usuário.

Os testes confirmaram que o escalonador distribui o tempo de CPU de forma proporcional ao número de tickets de cada processo. Os resultados obtidos (408:239:144 para uma razão esperada de 3:2:1) demonstram que o algoritmo funciona corretamente, com desvios dentro do esperado para um escalonador probabilístico.